

# GUVI Project 2 – Final Project Documentation

---

**React + TypeScript + Vite • Docker • AWS ECR • Amazon EC2 • Jenkins CI/CD**

*Project: Kanban Task Manager*

*Repository: [github.com/naviktg/Guvi-Project-2](https://github.com/naviktg/Guvi-Project-2)*

## 1. Project Overview

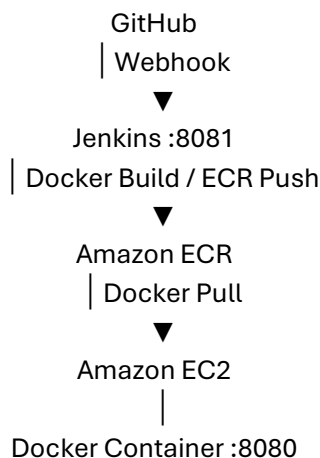
This project deploys a React + TypeScript + Vite Kanban Task Manager as a containerized application on Amazon EC2. Docker is used to build a production image with a multi-stage build and Nginx runtime. The image is stored in Amazon ECR. Jenkins automates the CI/CD workflow: source checkout, Docker build, ECR push, deployment to EC2, health validation, and cleanup.

## 2. Technology Stack

- Frontend: React, TypeScript, Vite
- Web server: Nginx
- Containerization: Docker
- Cloud: AWS
- Compute: Amazon EC2 (Amazon Linux 2023, t3.micro)
- Container Registry: Amazon ECR
- CI/CD: Jenkins 2.568.3
- Java runtime for Jenkins: Amazon Corretto 21
- Source control: Git and GitHub
- Monitoring/validation: Docker HEALTHCHECK and Jenkins pipeline validation

## 3. Deployment Architecture

Developer pushes code to GitHub → GitHub webhook triggers Jenkins → Jenkins checks out the repository → Docker image is built → image is pushed to Amazon ECR → EC2 pulls the latest image → old application container is replaced → health check validates the application.



## 4. Docker Implementation

The Dockerfile uses a two-stage build. The first stage uses Node 22 Alpine to install dependencies and create the Vite production build. The second stage uses Nginx Alpine to serve the generated static files.

- Multi-stage Docker build reduces the runtime image contents.
- Nginx serves the production application on port 8080.
- The container runs as a non-root user.
- Docker HEALTHCHECK calls the local application endpoint.
- .dockerignore excludes node\_modules, dist, Git metadata, environment files and editor files.
- Nginx is configured for SPA routing with: `try_files $uri $uri/ /index.html`.

## 5. AWS Implementation

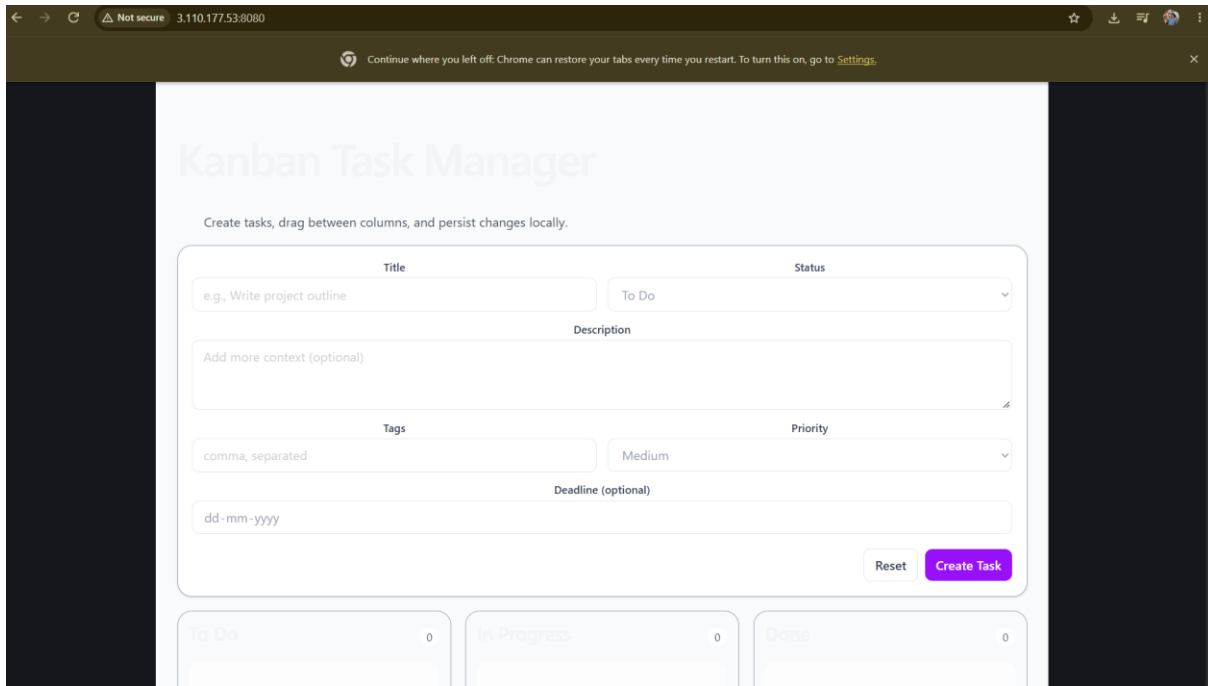
- Amazon EC2 hosts Jenkins and the deployed application container.
- Amazon ECR stores the container image guvi-project-2:latest.
- An EC2 IAM role provides AWS permissions used by the deployment workflow.
- Security Group rules allow the required application and Jenkins ports.
- Application is exposed on port 8080; Jenkins is configured on port 8081 because 8080 is used by the application.
- A 2 GiB swap file was configured on the small EC2 instance to provide additional memory headroom.

## 6. Jenkins CI/CD Pipeline

1. **Checkout:** Jenkins checks out the main branch from the GitHub repository.
2. **Build:** Docker builds the application image from the repository Dockerfile.
3. **ECR Login:** Jenkins authenticates Docker to Amazon ECR using AWS CLI credentials/EC2 IAM permissions.
4. **Push:** The image is pushed to the ECR repository.
5. **Deploy:** The EC2 application container is recreated from the latest ECR image.
6. **Health Check:** The pipeline validates that the application becomes healthy.
7. **Cleanup:** Unused Docker images are pruned where possible.
8. **Post Actions:** The pipeline prints a successful CI/CD deployment message.

## 7. CI/CD Validation Evidence

The completed Jenkins runs show successful end-to-end deployment. Build #2 and Build #3 finished with SUCCESS, and the pipeline output includes both 'Application is healthy!' and 'CI/CD deployment completed successfully!'. The EC2 container is also shown as '(healthy)'.



*Deployed Kanban Task Manager on EC2*

```
aws
[ec2-user@ip-172-31-4-221 ~]$ df -h /tmp
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p1  8.0G  3.1G  4.9G  39% /
tmpfs           457M  4.8M  452M   2% /tmp
[ec2-user@ip-172-31-4-221 ~]$ docker system df
TYPE        TOTAL    ACTIVE    SIZE      RECLAIMABLE
Images      2        2         48.85MB   0B (0%)
Containers  2        0         2B        2B (100%)
Local Volumes 0        0         0B        0B
Build Cache 21       0         11.35MB   11.35MB
[ec2-user@ip-172-31-4-221 ~]$ sudo fallocate -l 2G /swapfile
[ec2-user@ip-172-31-4-221 ~]$ sudo chmod 600 /swapfile
[ec2-user@ip-172-31-4-221 ~]$ sudo mkswap /swapfile
Setting up swapspace version 1, size = 2 GiB (2147479552 bytes)
no label, UUID=a750092e-8a44-4b07-adaf-670e91f9e37e
[ec2-user@ip-172-31-4-221 ~]$ sudo swapon /swapfile
[ec2-user@ip-172-31-4-221 ~]$ echo '/swapfile swap swap defaults 0 0' | sudo tee -a /etc/fstab
/swapfile swap swap defaults 0 0
[ec2-user@ip-172-31-4-221 ~]$ free -h
              total        used        free      shared  buff/cache   available
Mem:           913Mi       600Mi       63Mi        5.0Mi       248Mi       173Mi
Swap:          2.0Gi          0B        2.0Gi
[ec2-user@ip-172-31-4-221 ~]$ docker ps
CONTAINER ID   IMAGE                                NAMES      COMMAND                  CREATED    STATUS      PORTS
f74c42187f28   471443767061.dkr.ecr.ap-south-1.amazonaws.com/guvi-project-2:latest  "/docker-entrypoint..."  3 minutes ago  Up 3 minutes (healthy)  80/tcp, 0.0.0.0:80
[ec2-user@ip-172-31-4-221 ~]$ docker ps
CONTAINER ID   IMAGE                                NAMES      COMMAND                  CREATED    STATUS      PORTS
b78ba35459e1   471443767061.dkr.ecr.ap-south-1.amazonaws.com/guvi-project-2:latest  "/docker-entrypoint..."  3 minutes ago  Up 3 minutes (healthy)  80/tcp, 0.0.0.0:80
[ec2-user@ip-172-31-4-221 ~]$
```

*EC2 Docker container running and healthy*

 Jenkins

Guvi-Project-2-CI-CD

Search

Settings

User

Status

Changes

Build Now

Delete Pipeline

Configure

Rename

Stages

Pipeline Syntax

GitHub Hook Log

Builds >

Filter

Today

✓ #3 9:30 AM

✓ #2 9:13 AM

⌚ #1 8:32 AM

✓

Guvi-Project-2-CI-CD

Add description

Permalinks

- Last build (#3), 1 min 6 sec ago
- Last stable build (#3), 1 min 6 sec ago
- Last successful build (#3), 1 min 6 sec ago
- Last unsuccessful build (#1), 58 min ago
- Last completed build (#3), 1 min 6 sec ago

REST API

Jenkins 2.568.3

Jenkins job history showing successful builds

 Jenkins

Guvi-Project-2-CI-CD / #3

Search

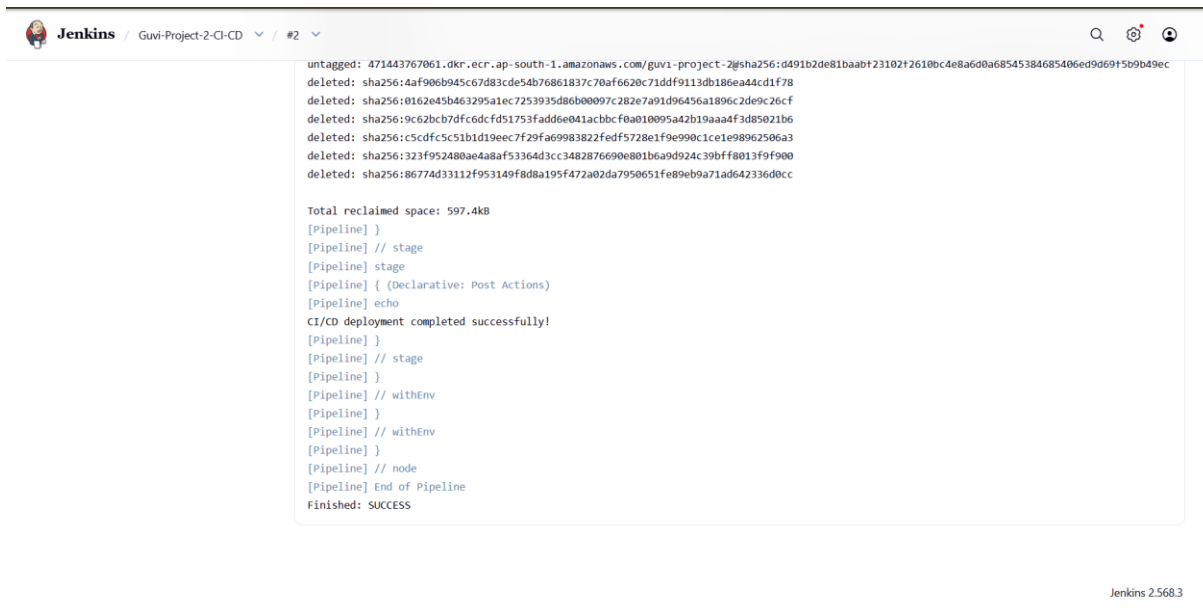
Settings

User

```
+ echo 'Application is healthy!'
Application is healthy!
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Cleanup)
[Pipeline] sh
+ docker image prune -f
Total reclaimed space: 0B
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions) (hide)
[Pipeline] echo
CI/CD deployment completed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Jenkins 2.568.3

Jenkins Build #3 console output ending in SUCCESS



```
untagged: 471443767061.dkr.ecr.ap-south-1.amazonaws.com/guvi-project-2@sha256:d491b2de81baabf23102f2610bc4e8a6d0a68545384685406ed9d69f5b9b49ec
deleted: sha256:4af906b945c67d83cde54b76861837c70af6620c71dd9113db186ea44cd1f78
deleted: sha256:0162e45b463295a1ec7253935d80b00097c282e7a91d96456a1896c2de9c26cf
deleted: sha256:9c62bcb7dfc6dcfd51753fadd6e041acbbcfa010095a42b19aaa4f3d85021b6
deleted: sha256:c5dcfc5c1b1d19eec7f29fa69983822fedf5728e1f9e990c1ce1e98962506a3
deleted: sha256:323f952480ae4a8af53364d3cc3482876690e001b6a9d924c39bfff013f9f900
deleted: sha256:b6774d33112f953149f8d8a195f472a02da7950651fe89eb9a71ad642336d0cc

Total reclaimed space: 597.4kB
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
CI/CD deployment completed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Jenkins 2.568.3

*Jenkins Build #2 console output ending in SUCCESS*

## 8. Final Verification Checklist

- ✓ Application accessible from EC2 public endpoint on port 8080 – PASS
- ✓ Docker container status is healthy – PASS
- ✓ GitHub repository contains Jenkinsfile – PASS
- ✓ Jenkins successfully checks out GitHub repository – PASS
- ✓ Docker image successfully builds – PASS
- ✓ Image successfully pushed to Amazon ECR – PASS
- ✓ EC2 successfully pulls and runs the image – PASS
- ✓ Application health check succeeds – PASS
- ✓ Jenkins Build #2 – SUCCESS
- ✓ Jenkins Build #3 – SUCCESS
- ✓ GitHub webhook configured and delivery reported successful – PASS

## 9. Interview / Project Explanation – 30 Second Answer

“I developed a Kanban Task Manager using React, TypeScript and Vite. I containerized the application using a multi-stage Docker build and served the production build with Nginx. I created an ECR repository in AWS and deployed the container on an EC2 instance. For CI/CD, I configured Jenkins with a Jenkinsfile and GitHub webhook. Whenever code is pushed, Jenkins checks out the code, builds the Docker image, pushes it to ECR, deploys the latest image on EC2, and verifies the container health. The pipeline has been successfully tested with Jenkins builds #2 and #3.”

## 10. Interview Questions and Suggested Answers

### **Q: What is the project about?**

A: It is a Kanban Task Manager deployed as a Docker container on AWS EC2, with Jenkins automating the CI/CD workflow from GitHub to ECR and EC2.

### **Q: Why did you use Docker?**

A: Docker packages the application and its runtime configuration into a consistent image, so the same image can be built, stored and deployed without depending on the host's Node environment.

### **Q: Why did you use a multi-stage Dockerfile?**

A: The build stage creates the production frontend files, while the runtime stage only needs Nginx and the generated files. This keeps the runtime image cleaner and smaller.

### **Q: Why Nginx?**

A: The Vite production output is static content. Nginx is suitable for serving static files efficiently and also handles SPA fallback routing.

### **Q: Why Amazon ECR?**

A: ECR provides a private AWS container registry where the Docker image can be stored and pulled by the EC2 deployment environment.

### **Q: Why Jenkins?**

A: Jenkins automates the repetitive CI/CD steps so a GitHub change can trigger checkout, build, image push, deployment and health validation.

### **Q: How does the GitHub webhook work?**

A: GitHub sends an HTTP POST event to the Jenkins GitHub webhook endpoint when a configured repository event occurs. Jenkins then triggers the pipeline.

### **Q: Why is Jenkins on port 8081?**

A: The application already uses port 8080 on the same EC2 instance, so Jenkins was configured on 8081 to avoid a port conflict.

### **Q: How do you know the deployment succeeded?**

A: Jenkins Build #2 and #3 ended with Finished: SUCCESS, the pipeline printed 'Application is healthy!', and the EC2 Docker container showed the healthy status.

### **Q: What happens when new code is pushed?**

A: GitHub sends the webhook, Jenkins checks out the new revision, builds a new Docker image, pushes it to ECR, deploys the latest image to EC2, and validates application health.

### **Q: What is the purpose of the IAM role?**

A: It gives the EC2 environment AWS permissions without putting long-lived AWS access keys inside the application or Jenkinsfile.

**Q: What is the purpose of the Docker HEALTHCHECK?**

A: It allows Docker to determine whether the application inside the container is responding correctly, rather than only checking whether the process is running.

**Q: Why did you add swap?**

A: The EC2 instance is a small t3.micro with limited RAM. A 2 GiB swap file provides additional memory headroom during resource-intensive operations such as Docker builds.

**Q: What was one issue you faced?**

A: Jenkins initially could not start because its default port 8080 conflicted with the application container. I moved Jenkins to port 8081 and updated the Security Group accordingly.

**Q: What was another issue?**

A: The initial Jenkins build was aborted/hung while resources were constrained. I checked disk, memory, Docker usage and added swap before rerunning the pipeline successfully.

**Q: How would you improve this project for production?**

A: I would use HTTPS with a domain/reverse proxy, separate Jenkins from the application host, use dedicated Jenkins agents, apply least-privilege IAM, use immutable image tags, add centralized logging/monitoring, and deploy behind a load balancer.

## 11. Important Interview Tip

Do not try to memorize every command. Explain the flow clearly: GitHub → Jenkins → Docker → ECR → EC2 → Health Check. If the interviewer asks for a command, give the relevant command and explain what it does. If asked about a design improvement, distinguish the current learning/demo architecture from a production architecture.

## 12. Final Status

**Core implementation and CI/CD workflow: COMPLETED**

The project has demonstrated a working automated deployment path and successful health validation. Remaining work, if required by the submission process, is administrative/documentation packaging rather than a missing core deployment component.